TP1 — Auditer un dépôt Git : où sont les données personnelles ?

Sommaire

1. Objectifs . . . 1
2. Prérequis . . . 2
3. Contexte . . . 2
4. Étape 1 — Récupérer le dépôt à auditer . . . 2
5. Étape 2 — Que contient **vraiment** ce dépôt ? . . . 3
   5.1. 2.1 L'état actuel . . . 3
   5.2. 2.2 L'historique : tous les fichiers qui ont **un jour** existé . . . 3
   5.3. 2.3 Qui a fait quoi, et quand ? . . . 3
   5.4. 2.4 Chercher une donnée dans tout l'historique . . . 3
6. Étape 3 — Classer les données trouvées (grille du TD1) . . . 4
7. Étape 4 — « Supprimé » ne veut pas dire « effacé » . . . 5
8. Étape 5 — Empêcher la fuite plutôt que la réparer . . . 6
9. Étape 6 — Rendre votre audit . . . 7
10. Livrable . . . 7
11. Mémo des commandes d'audit . . . 8

Le TD1 vous a appris à **classer** une donnée sur le papier. Ce TP vous fait faire la même chose sur un **vrai dépôt Git**, celui d'une application interne de **MyCallCenter**. Vous allez y découvrir des données personnelles — et surtout constater qu'un fichier **supprimé** d'un dépôt Git n'est **pas effacé** pour autant.

1. Objectifs

 À la fin de ce TP, vous saurez :

- **retrouver** les données personnelles présentes dans un dépôt et dans son **historique** ;
- les **classer** avec la grille du TD1 (NP / DIR / IND / SENS) ;
- démontrer qu'un `git rm` ne fait **pas** disparaître une donnée de l'historique, et en tirer les conséquences pour le **droit à l'effacement** ;
- mettre en place un `.gitignore` qui **empêche** la fuite au lieu de la réparer.

 Durée : **1 heure**.

2. Prérequis

- **TP0 — Introduction à Git** fait (dépôt, commit, log, `.gitignore`, push GitHub).
- **Cours 1** et **TD1 — Classer les données** faits.
- **Git Bash** ouvert.

3. Contexte

Vous êtes en stage au pôle **Données** de **MyCallCenter**. Le **DPO** vous signale que l'application interne **MyCom** a été développée « vite fait » par deux développeurs. Avant sa mise en production, il veut savoir :

> « Est-ce que des données personnelles se sont retrouvées dans le dépôt Git de MyCom ? Et si oui, peut-on encore les récupérer ? »

C'est votre mission.



Toutes les données du dépôt d'exercice sont **fictives**. Elles imitent des données réelles pour que l'exercice ait du sens, mais ne concernent aucune personne existante.

4. Étape 1 — Récupérer le dépôt à auditer

□□ ~5 min

Le dépôt vous est fourni sous forme d'un **script de fabrication** : il reconstruit à l'identique, sur votre poste, le dépôt de MyCom avec tout son historique.

Dans **Git Bash**, placez-vous dans le dossier `scripts/` de la séance, puis :

```
bash fabrique-depot-audit.sh
cd ~/Documents/mycom-audit
```

Vérifiez que vous êtes bien dans un dépôt de 6 commits :

```
git log --oneline
```



Si le script répond « Le dossier existe déjà », c'est que vous l'avez déjà lancé. Passez directement au `cd`.

5. Étape 2 — Que contient vraiment ce dépôt ?

□□ ~10 min

5.1. 2.1 L'état actuel

```
git ls-files
```

`git ls-files` liste les fichiers suivis par Git **à l'instant présent**. Notez-les.

a. Un fichier de cette liste vous paraît-il **anormal** dans un dépôt de code ? Ouvrez-le et dites pourquoi.

5.2. 2.2 L'historique : tous les fichiers qui ont un jour existé

Un dépôt ne contient pas que son dernier état. Demandons à Git **tous** les fichiers ajoutés depuis le début :

```
git log --diff-filter=A --name-only --pretty=format:''
```

a. Comparez avec la liste de l'étape 2.1. **Quel fichier a disparu** de l'état actuel tout en figurant dans l'historique ?

5.3. 2.3 Qui a fait quoi, et quand ?

```
git log --pretty=format:'%h | %an | %ad | %s' --date=short
```

a. Lisez les **messages de commit**. Lequel ajoute le fichier suspect ? Lequel prétend le retirer ?

b. **Qui** (nom, e-mail) a introduit ce fichier, et à quelle date ?



Ce que vous êtes en train de faire

Vous exploitez la **traçabilité** de Git : l'historique dit **qui** a fait **quoi** et **quand**. C'est exactement le pilier **P (Preuve)** que nous verrons au Cours 4. Sans cet historique, l'audit du DPO serait impossible.

5.4. 2.4 Chercher une donnée dans tout l'historique

Git sait retrouver **à quel commit** un mot est apparu ou a disparu (option `-S`, dite « la pioche ») :

```
git log --oneline -S "num_secu"
git log --oneline -S "DB_PASSWORD"
```

a. Combien de commits sont concernés dans chaque cas ? Que vous apprend le **deuxième** résultat sur le fichier de configuration ?

6. Étape 3 — Classer les données trouvées (grille du TD1)

□□ ~15 min

Affichez le contenu du fichier disparu, en le lisant **directement dans l'historique** (remplacez `HASH` par l'identifiant du commit qui l'a ajouté, trouvé à l'étape 2.3) :

```
git show HASH:data/export-clients.csv
```

Le fichier s'affiche **intégralement**, alors qu'il n'existe plus dans le dossier. Gardez cette observation : elle est le cœur de l'étape 4.

Travail à faire. Pour **chaque colonne** du fichier, remplissez le tableau ci-dessous avec la grille du TD1 : **NP** (non personnelle), **DIR** (identification directe), **IND** (identification indirecte), **SENS** (sensible, art. 9).

Colonne	Catégorie	Justification (une phrase)
`id`		
`nom`, `prenom`		
`email`		
`code_postal`		
`ip_connexion`		
`id_cookie`		
`num_secu`		
`carte_bancaire`		
`commentaire_libre`		



Le champ libre : lisez-le ligne par ligne

La colonne `commentaire_libre` n'a pas **une seule** catégorie : tout dépend de ce que la personne a écrit. Examinez les **cinq lignes** du fichier et indiquez, pour chacune, si le commentaire révèle une donnée **sensible** — et quelle catégorie de l'article 9 elle touche.

C'est le piège du Cours 1 : **un formulaire anodin peut collecter des données sensibles sans l'avoir prévu**.

a. Regardez maintenant `data/jeu-de-test.csv`, le fichier qui a **remplacé** l'export. Contient-il des données personnelles ? Justifiez avec le vocabulaire du Cours 1 (**anonyme** / **pseudonyme** / **agrégé**).

7. Étape 4 — « Supprimé » ne veut pas dire « effacé »

□□ ~10 min

Le commit `Retire l□export clients` semble avoir réglé le problème : le fichier n'est plus là. Vérifions.

```
ls data/
git log --oneline --full-history -- data/export-clients.csv
```

Le fichier est absent du dossier, mais Git connaît encore **les deux commits** de sa vie : celui qui l'a créé et celui qui l'a supprimé. Et surtout, il sait le **restituer** :

```
git show HASH:data/export-clients.csv > ~/fuite-recuperee.csv
wc -l ~/fuite-recuperee.csv
```

Les données sont revenues, intactes : **6 lignes**, l'en-tête et les cinq personnes.



Vous venez de créer, sur le disque du poste, un fichier contenant des données personnelles. Ne le laissez pas traîner — c'est exactement le geste qui est à l'origine de la fuite que vous auditez. Supprimez-le dès que vous l'avez compté :

```
rm ~/fuite-recuperee.csv
```

Notez qu'il a été écrit **hors du dépôt** (`~/`) : produire ce fichier **dans** le dossier `mycom-audit` aurait risqué de le voir repartir dans un commit.



La conclusion RGPD de ce TP

Un dépôt Git est conçu pour être **indélébile** : c'est sa qualité (on peut toujours revenir en arrière) et, ici, son **danger**.

- Une donnée personnelle commitée est dans l'historique **pour toujours**, même « supprimée » par un commit ultérieur.
- Si le dépôt a été **poussé** sur une forge (GitHub, GitLab), chaque personne qui l'a **cloné** possède une copie complète de l'historique — donc de la fuite.
- Le **droit à l'effacement** (article 17 du RGPD) devient alors très difficile à honorer : il faut **réécrire l'historique** du dépôt et demander à tous les clones de repartir de zéro.

a. Une personne figurant dans le fichier exerce son **droit à l'effacement**. En une ou deux phrases : pourquoi la demande est-elle **techniquement compliquée** à satisfaire ici, et que faudrait-il faire ?

b. Le commit qui « retire » l'export porte le message `Retire l'export clients (donnees reelles, alerte du DPO)`. En quoi ce message est-il **trompeur** pour qui lirait seulement le `git log` ?

8. Étape 5 — Empêcher la fuite plutôt que la réparer

~10 min

La seule parade efficace est **en amont** : ne jamais laisser ces fichiers entrer dans un commit.

1. Créez le garde-fou à la racine du dépôt. Tapez ces quatre commandes :

   ```
   echo ".env"                 >  .gitignore
   echo "config/.env"          >> .gitignore
   echo "data/export-*.csv"    >> .gitignore
   echo "*.dump"               >> .gitignore
   ```

   Vérifiez le résultat avec `cat .gitignore` : le fichier doit contenir quatre lignes.

2. Retirez du suivi le fichier de secrets, **sans le supprimer de votre disque** :

   ```
   git rm --cached config/.env
   git add .gitignore
   git commit -m "Ajoute un .gitignore et retire les secrets du suivi"
   git status
   ```

3. Testez le garde-fou : recréez un export interdit et regardez ce que Git propose.

   ```
   echo "nom;email" > data/export-clients.csv
   git status
   ```

a. Git propose-t-il ce nouveau fichier ? Pourquoi ?

b. Relancez `git log --oneline -S "DB_PASSWORD"`. Le mot de passe est-il, **lui**, sorti de l'historique ? Que faut-il faire **en plus**, dans la vraie vie, pour un secret qui a déjà été poussé sur une forge ?



Réponse attendue à la dernière question : le `.gitignore` protège l'**avenir**, pas le **passé**. Un secret déjà commité doit être considéré comme **compromis** : la seule mesure sérieuse est de le **changer** (nouveau mot de passe, nouveau jeton), en plus de nettoyer l'historique.

9. Étape 6 — Rendre votre audit

~10 min

Vous allez créer le dépôt qui vous servira **toute l'année** pour ce Bloc 3.



Ce qu'il ne faut surtout pas faire

Ne poussez jamais `mycom-audit` sur GitHub. Ce serait reproduire exactement la faute que vous venez d'auditer : publier un historique contenant des données personnelles. Votre rendu est un **rapport**, pas le dépôt fautif.

1. Créez un dépôt `bloc3-cyber` sur GitHub (**Public**, sans README), puis en local :

```
cd ~/Documents
mkdir -p bloc3-cyber/tp1-rgpd
cd bloc3-cyber
git init
git remote add origin https://github.com/VOTRE-USERNAME/bloc3-cyber.git
```

2. Dans `tp1-rgpd/`, déposez deux fichiers :
   - `classification.csv` — le tableau de l'étape 3, une ligne par colonne auditée, au format `colonne;categorie;justification` ;
   - `rapport-audit.md` — 15 à 25 lignes répondant aux questions des étapes 2, 4 et 5.



Votre rapport **décrit** la fuite, il ne la **recopie pas** : citez les **colonnes** et les **catégories**, jamais les valeurs (aucun nom, aucun numéro, aucun mot de passe). Un rapport d'audit qui recopie les données est lui-même une fuite.

3. Commitez et poussez :

```
git add .
git commit -m "TP1 : audit RGPD du depot MyCom"
git push -u origin main
```

4. Sur GitHub : **Settings** → **Collaborators** → **[Add people]** → `timotirob`.

10. Livrable

À rendre :

- ☐ Le dépôt `bloc3-cyber` sur GitHub, enseignant ajouté en collaborateur.
- ☐ `tp1-rgpd/classification.csv` : les 9 colonnes classées, et les 5 commentaires libres analysés.
- ☐ `tp1-rgpd/rapport-audit.md` : les fichiers trouvés, qui les a introduits et quand, la démonstration

que la suppression n'efface pas, et les mesures de prévention.

Table 1. Auto-évaluation avant de rendre

Vérification	OK ?
J'ai retrouvé le fichier supprimé **et** le fichier de secrets	[ ]
J'ai classé chaque colonne avec la grille NP / DIR / IND / SENS	[ ]
J'ai repéré les données **sensibles** cachées dans le champ libre	[ ]
Je sais expliquer pourquoi `git rm` n'efface pas une donnée	[ ]
Mon rapport ne recopie **aucune** valeur personnelle	[ ]
Je n'ai **pas** poussé `mycom-audit` sur GitHub	[ ]

11. Mémo des commandes d'audit

Commande	Rôle
`git ls-files`	Fichiers suivis **actuellement**
`git log --diff-filter=A --name-only`	Tous les fichiers **un jour** ajoutés
`git log --pretty=format:'%h %an %ad %s'`	Qui a fait quoi, et quand
`git log -S "motif"`	Commits où un motif apparaît ou disparaît (« la pioche »)
`git log --full-history -- <fichier>`	Toute la vie d'un fichier, même supprimé
`git show <hash>:<fichier>`	Afficher un fichier **tel qu'il était** à un commit
`git rm --cached <fichier>`	Retirer du suivi sans supprimer du disque

BTS SIO - ENC Paris — Bloc 3 - Cybersécurité des services informatiques — 2026-2027